

Algoritmer och Datastrukturer

Extratillfälle

Tentamen

Information om tentan

- E-tentamen som skrivs i Inspera
- Eftersom Inspera är byggt av klåpare kommer vi inte kunna kompilera koden som jag hade tänkt mig, och vi kan därför inte köra en tentamen där ni förväntas skriva lika mycket kod
- Det finns dock ett editorläge som gör det lättare att skriva kod än i ordbehandlarläget
- Ni kommer förväntas kunna fylla i t.ex. En implementation för en enklare metod, men ni kommer inte behöva göra ett program med flera olika metoder/klasser som hänger samman eller liknande

Information om tentan

- Kommer bifoga en pdf med tangentbordskommandon för de som är vana vid att programmera på Mac
- Alternativt kanske lägga en hög med papper i tentasalen; får se vad studieadmin tycker
- Ingen ska behöva känna panik över att använda en lånad dator

Vägledning för hur man skapar kodklamrar och andra specialtecken på Windowsdatorer
(hjälpamt för de som brukar programmera på Mac)

Shift + 7 för /
Shift + 8 för (
Shift + 9 för)

Alt Gr + 7 för {
Alt Gr + 0 för }
Alt Gr + 8 för [
Alt GR + 9 för]

Alt Gr och + för \

Information om tentan

- Det **bästa sättet att vara förberedd för en tenta är att hårdplugga föreläsningsmaterialet**: jag har tagit upp de saker jag tycker är viktiga för kursens omfattning
- Om jag inte gått genom någonting behöver ni inte bry er om det: datastrukturer och sorteringar som vi inte pratat om kommer inte att dyka upp på examinationen
- Läs presentationerna flera gånger, kolla genom övningsuppgifterna i studium, kolla på duggorna, på gamla tentor, och kolla genom labbarna från kursen
- Repot som vi arbetat med under kursen innehåller allt material - kolla på videoklippen och läs länkarna, kolla genom kodexemplen som vi skapat på sal, osv
- Det kommer **INTE** att komma några frågor om testning och Test-Driven Design eftersom dessa koncept inte finns med i kursmålen

Information om rättning av projektet

- Tack för trevliga seminarium i går!
- Vi kommer att sätta oss ner tillsammans nästa vecka och kika på inlämningarna gemensamt och prata om hur pass aktiva och pålästa folk var under seminariet
- **Alla kommer få lite komplettering**, och det är inte för att ni gjort dåliga projekt utan för att det alltid finns lite småsaker att rätta till: **tanken är att det ska vara ett lärandetillfälle**
- Ni kommer få utförlig skriftlig feedback eftersom det är den typ av feedback som vi själva vill ha: om man lagt ner tid och möda på en uppgift känns det rimligt att läraren också lägger ner tid på att utvärdera lösningen ordentligt
- Förhoppningen är att alla hunnit få feedbacken till nästa fredag
- Om vi bedömer att du kan komplettera rapport och projekt upp till godkänt kommer du ha till den 31 mars på dig, men det är en hård deadline
- Missar du den får du göra om seminariet

Repetition: tidskomplexiteter

När vi pratar om “n” menar vi alltid datamängden som vi opererar på i algoritmen:

Skickar vi in en lista är n antalet värden i listan, osv

Om tiden:

... är det:

... som betecknas:

Dubblas när datamängden dubblas

Fyrdubblas när datamängden dubblas

Åttadubblas när datamängden dubblas

Dubblas varje gång **n ökar med 1**

Linjär tidskomplexitet

Kvadratisk tidskomplexitet

Kubisk tidskomplexitet

Exponentiell tidskomplexitet

$O(n)$

$O(n^2)$

$O(n^3)$

$O(2^n)$



Vi gör skillnad mellan dessa eftersom de är olika grader av hemskhet 🐱 $O(n^2)$ är ofta en giltig komplexitet för mindre n

Om ni däremot får tidskomplexitet som är $O(n^3)$ (eller värre) bör ni försöka konstruera en bättre lösning, för den är i princip oanvändbar

Tidskomplexitet och platskomplexitet

- **Tidskomplexitet** är ett mått på **hur tiden utvecklas** när indatan i en algoritm ökar
- **Platskomplexitet** är ett mått på **hur minneseffektiv en algoritm är**: hur mycket plats i RAM-minnet upptar den?
- Algoritmer med **dålig platskomplexitet** är ofta **oönskvärda** även om de kan ha bra tidskomplexitet eftersom **minne är en dyr resurs** i datorer
- **Undvik att kopiera** över saker i mellanliggande listor och liknande om möjligt, särskilt när datamängden är stor
- Om en sorteringsalgoritm är **In-Place** innebär det att den inte skapar några nya datastrukturer för att sortera datan. **Vi säger att den har $O(1)$ i platskomplexitet**

ADT: abstrakt datatyp

- En ADT är en Abstrakt DataTyp, dvs en datatyp som **inte definieras av en specifik implementation** (vi menar kod när vi säger implementation) **utan av ett beteende**
- Den kan vara **skriven på flera olika sätt** och använda olika data-strukturer “under huven” så länge den **uppfyller specifika krav** på hur den beter sig
- En **Lista ska till exempelvis vara dynamisk**, till skillnad från en simpel array: vi ska alltid kunna stoppa in fler värden i den utan att få slut på utrymme
- **Exempel:** Både **ArrayList** och **LinkedList** är olika sätt att implementera en Lista (de har sina egna implementationer av de metoder som Javas List-interface säger att de måste innehålla)

Vanliga abstrakta datatyper

ADT	Definition	Implementationer i Javas standardbibliotek:
Lista	Ordnad samling av element	ArrayList, LinkedList
Stack	LIFO: Last in, First Out	ArrayDeque
Kö	FIFO: First in, First Out	ArrayDeque, PriorityQueue
Träd	Hierarkiskt lagrad data	TreeSet (Red and black tree)
Map	Lagrar nyckel-värde-element	TreeMap, HashMap, LinkedHashMap
Graf	Lagrar relationer mellan data	- (Använd tredjepartsbibliotek som t.ex. Jgraph som finns gratis på Github)

– Det finns fler, men dessa är de som vi gått genom på kursen. Ni behöver inte känna till några andra

Jämförelse av datastrukturer

	Lista Samtliga	Lägga till	Ta bort	Söka	Intervall
Array	$O(n)$	$O(n)$	$O(n)$	$O(n)$	$O(k)$
ArrayList	$O(n)$	$O(1)$	$O(n)$	$O(n)$	$O(k)$
LinkedList	$O(n)$	$O(1)$	$O(1)$	$O(n)$	$(n+k)$
TreeMap	$O(n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(1)$
HashMap	$O(n)$	$O(1)$	$O(1)$	$O(1)$	$O(k)$

Jämförelse, olika sorteringsalgoritmer

Algoritm	Tidskomplexitet (Bäst/Sämst)	In-Place	Stabil	Används
QuickSort	$O(n \cdot \log n)$ / $O(n^2)$	Ja	Nej	Stora dataset som behöver in-place-sortering
MergeSort	$O(n \cdot \log n)$	Nej	Ja	När stabilitet är viktigt
InsertionSort	$O(n)$ / $O(n^2)$	Ja	Ja	Små eller mestadels sorterade samlingar
BubbleSort	$O(n^2)$	Ja	Ja	Helst inte, men ok för små dataset och naiva lösningar
SelectionSort	$O(n^2)$	Ja	Nej	Används sällan men föredras över BubbleSort pga färre swaps



Bra algoritmer



Används främst i lärosyfte

Sortering: Array eller lista?

- **Arrays.sort()** är till för arrayer. Den kan sortera arrayer som innehåller:
 - * Primitiva typer: QuickSort används
 - * Objekt: TimSort används
- **Collections.sort()** är till för listor. Den är alltid stabil men aldrig in-place. Exempel på listor som den kan sortera:
 - * ArrayList
 - * LinkedList
- Collections.sort() **funkar bara för listor**. Vill man sortera t.ex. en **HashMap** efter nycklar eller värden brukar man:
 1. **Konvertera** den till en List<Map.Entry<K, V>> och sortera listan, eller:
 2. Konvertera den **till en TreeMap**, som då sorterar automatiskt efter nycklar

Rekursivitet

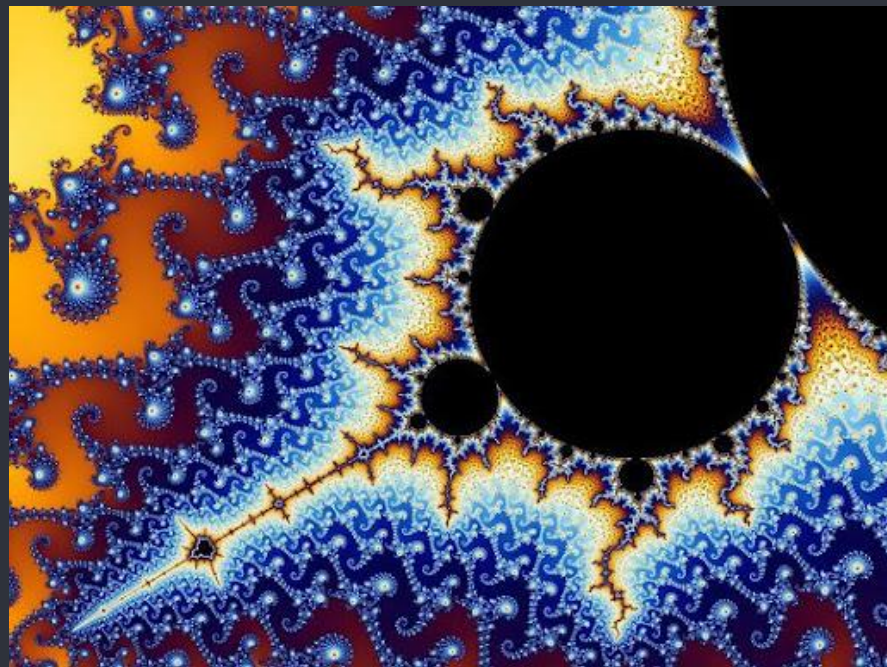
- **Rekursion** är när någonting definieras i termer av sig självt. I programmering menar vi:
 - * **En metod** som anropar sig själv (**rekursiv algoritm**)
 - * **En klass** som innehåller instanser av sig själv (**rekursiv datastruktur**)
- Rekursion finns överallt, inte bara i programmering. Naturen och matematiken är uppbyggda av självrefererande system
- Exempel på rekursiva algoritmer: MergeSort, binär sökning, fibonacci - divide-and-conquer-problem är ofta unikt lämpade för rekursion
- Exempel på datastrukturer som är rekursiva: Länkade listor, binära träd

Rekursivitet

- **Rekursion använder sig av callstacken:** varje nytt anrop lägger en ny stackframe på callstacken och när en metod **nått basfallet** börjar stacken att lindas upp
- Det är därför rekursiva algoritmer och strukturer **har ett backtrack-
ingbeteende**
- **Kan skapa ett Stack Overflow** om de saknar ett basfall eftersom de då fortsätter att lägga på stacken i all oändlighet

```
public long fib(int n)
{
    if(n <= 1) return n;
    return fib(n-1) + fib(n-2);
}
```

basfall



Algoritmdesigntechniker

- Det finns lite olika algoritmdesigntechniker man kan använda. Designtechnik och “paradigm” syftar till samma sak: det är bara namn som beskriver ett beteende. Några vanliga exempel är:

Teknik	Definition:	Exempel:
Brute force	Testar alla möjliga kombinationer (ineffektiv men ibland nödvändig)	Nästlade for-loopar
Divide-and-conquer	Delar upp ett problem i delproblem (naturligt lämpade för rekursion)	MergeSort, Quick-Sort, Binär sök
Greedy	Tar alltid det lokalt bästa beslutet (Ändrar sig aldrig)	Dijkstras algoritm
Backtracking	Utforskar alla möjliga vägar och kan ångra felaktiga beslut (mer effektiv än en brute force-approach)	Djupet-först i graf och pathfinding

Grafer

- En graf är en datastruktur som man använder när relationerna mellan datan är lika viktiga att lagra som datan själv
- Precis som träd, listor, stackar och köer är det en abstrakt data-typ som kan skapas på olika vis: den definieras utifrån vad den gör och inte hur den är implementerad
- Grafer är inte särskilt värdefulla för att lagra linjär data: det vill säga primitiva datatyper (typ integers), objekt, nyckel-värde-par, och liknande. Man använder dem bara om det finns meningsfulla relationer mellan dessa nummer eller objekt
- Sociala nätverk, Spotify, Netflix, osv använder sig ofta av grafdatabaser för att lagra relationer som du har till olika datapunkter: vilka filmer/låtar gillar du, vilka andra användare följer du, vilka följer dig, etc

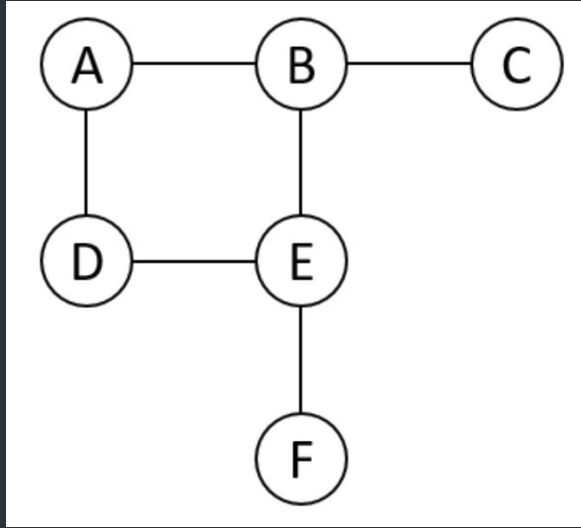
Att lagra grafer

- Lagras i grannmatriser eller grannlistor (adjacency matrix eller adjacency list)
- Vilken man använder beror på lite hur datan ser ut: om varje nod har många relationer till andra noder kan en matris vara lämplig
- Om de flesta noder har få relationer till andra noder blir det dock väldigt många nollor som vi lagrar i en matris, och då passar en grannlista bättre (mer minneseffektivt)
- För grannmatriser brukar vi använda tvådimensionella arrayer, och de har $O(n^2)$ i platskomplexitet: Om n är antalet noder så kommer vi skapa en array med storleken:

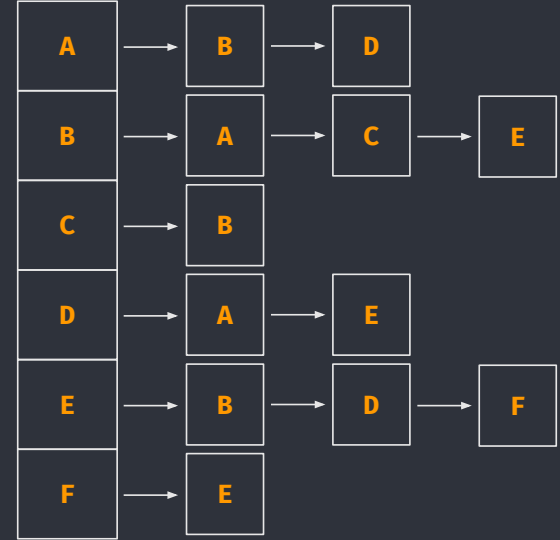
```
int[][] graph = new int[n][n];
```

- För 10 noder kommer vi behöva 100 celler i en matris. För 1000 noder kommer vi behöva $1000 * 1000 = 1000\ 000$

Grannmatrix och grannlista



	A	B	C	D	E	F
A	0	1	0	1	0	0
B	1	0	1	0	1	0
C	0	1	0	0	0	0
D	1	0	0	0	1	0
E	0	1	0	1	0	1
F	0	0	0	0	1	0



Genomgång av gamla tentamenfrågor

ADT (VT23)

Fråga: Du har en okänd datastruktur och du vill ta reda på vilken datastruktur det är. För att undersöka saken lagrar du i tur och ordning följande data i din datastruktur:

2, 4, 6, 7, 10, 12, 10, 8, 6, 4, 2

När du hämtar ut data från din datastruktur får du, i tur och ordning:

2, 4, 6, 8, 10, 12, 10, 7, 6, 4, 2.

Vilken datastruktur är det?

2, 4, 6, **8**, 10, 12, 10, **7**, 6, 4, 2. // Vi inser att 8 och 7 har bytt plats men i övrigt har datapunkterna samma ordning (i och för sig är det samma ordning både framifrån och bakifrån). Datastrukturen har alltså vänt på ordningen.

Lösning: ADT (VT23)

Kandidater: Lista, HashMap, TreeMap, Kö, Stack

Beteende: Vi tycks få ut datan i exakt omvänd ordning mot hur vi stoppade in den.

Maps lagrar nyckel-värde-typer så vi kan exkludera dem.

Listor lagrar värden i den ordning vi stoppar in dem. Träd sorterar datan automatiskt åt oss. En HashMap lagrar ingenting i en särskild ordning alls. Det kan inte vara någon av dessa.

En kö fungerar enligt principen FIFO: **First in, first out**. Vi borde med andra ord fått ut värdena i samma ordning som vi stoppade in dem.

En stack däremot fungerar enligt LIFO-principen: **Last in, first out!** Eftersom vi staplar ("stackar") saker på hög måste vi börja ta av dem från toppen igen när vi vill hämta ut dem.

Svar: Den abstrakta datatypen **borde vara en stack**, eftersom vi får ut värdena i omvänd ordning.

ADT (HT23)

Fråga: Följande operationer görs på en inledningsvis tom kö:

`enqueue(5), enqueue(8), dequeue(), enqueue(4), enqueue(3), size()`

Vilket data får vi när vi nu utför operationen `dequeue()`?

- En kö fungerar enligt principen **FIFO: First In, First Out**
- Operationerna gör följande: `enqueue()` lägger på kön, `dequeue()` hämtar ut från kön, `size()` ger oss hur många element kön innehåller men förändrar inte kön
- Vi lägger 5 och 8 i kön. Vi tar sedan bort det första elementet (5). 8 är nu först i kön. Vi lägger till 4 i kön, och sen 3. Kön innehåller 8, 4 och 3. Vi anropar `size()` som ger oss storleken.

Svar: Kön innehåller 8, 4, 3. Om vi hämtar ut ett värde från den kommer vi att få ut 8 eftersom det nu är först i kön.

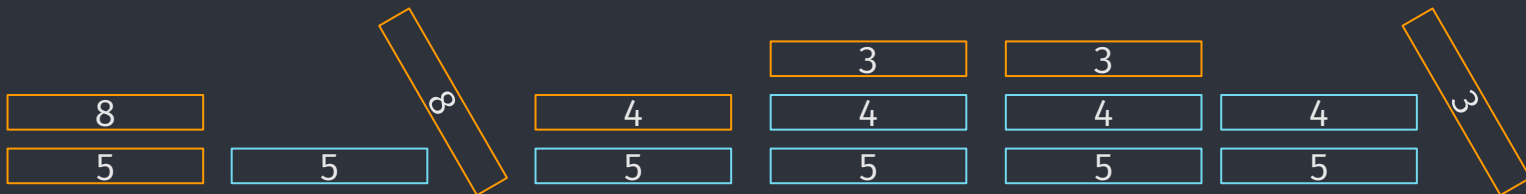
ADT 2 (2023-05-11)

Fråga: Följande operationer görs på en inledningsvis tom stack:
push(5), push(8), pop(), push(4), push(3), peek()

Vilket data får vi när vi nu utför operationen pop()?

8 tas bort eftersom pop returnerar och tar bort det översta elementet på stacken. Peek kollar bara på det översta elementet utan att ta bort det.

Svar: 3



Retur och tid 2 (2023-05-11)

Utifrån koden nedan:

1. Vad returnerar metoden om den anropas med `a=[8, 1, 3, 6, 4, 9, 1, 7]`?
2. Vilken tidskomplexitet har metoden (där `n` är längden på `a`)? Du ska även motivera varför metoden har denna tidskomplexitet.

```
public int summarize(int[] a)
{
    int sum = 0;
    for(int i = 0; i < a.length; i += 5) // O(n/5)
    { // Kommer köras varv 0 och 5 - alltså 2 ggr
        for(int j = 0; j < a.length; j += 3) // O(n/3)
        { // Kommer köras varv 0, 3, 6 (3 ggr)
            sum += a[i];
        }
    }
    return sum;
}
```

Koden returnerar **51** eftersom vi tar 3 gånger vad som finns på indexplats 0 och 5

$O(n/5) * O(n/3) \approx O(n^2)$ eftersom vi förenklar bort divisionerna

Trolig tidskomplexitet (2024-05-15)

Fråga: Vad är den troliga tidskomplexiteten för metoden vars tidstestning visas i tabellen nedan?

Motivera ditt svar.

<u>n</u>	<u>tid</u>
2	6
3	7
4	10
5	12
6	14

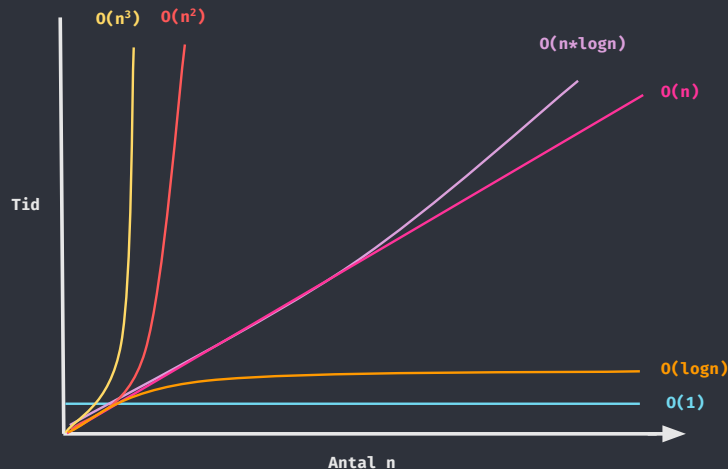
Svar: Tiden dubblas när antalet element dubblas så $O(2*n)$ vilket förenklas till $O(n)$

Trolig tidskomplexitet 3 (2025-03-19)

Fråga: Vad är den troliga tidskomplexiteten för metoden vars tidstestning visas i tabellen nedan? (6p)

Motivera ditt svar.

<u>n</u>	<u>tid</u>
2	1
4	7
8	28
16	144
32	300



Svar: Det går inte att läsa ut något tydligt mönster för vad som händer med tiden när n dubblas men vi kan läsa ut att den inte dubblas, i alla fall inte för de första värdena. För större värden verkar algoritmen närma sig $O(n)$. Möjligen är det $O(n \log n)$ men det är svårt att veta

Tidskomplexitet 4 (2025-05-14)

Fråga: Vad har metoden för tidskomplexitet i bästa och värsta fallet? Motivera ditt svar. (6 p)

```
/**
 * Returns true if elements are sorted in ascending order (from smallest to largest).
 *
 * @param a the array we want to check
 * @param index where to start, use a.length to check all elements
 * @return true if elements in 'a' are sorted in ascending order
 */
public boolean isSorted(int[] a, int index) {
    if(a.length <= 1 || index <= 1) {
        return true;
    }
    if(a[index - 1] >= a[index - 2]) {
        return isSorted(a, index - 1);
    }
    return false;
}
```

Ex:
int[] array = {7, 1, 9};
isSorted(array, array.length);

isSorted(array, array.length-1); // 2 vilket ger
false eftersom array[1] < array[0]
isSorted(array, array.length); // 3

Svar: Ju mer sorterad den är från slutet, desto fler element att kolla, så desto fler lookups (närmar sig $O(n)$). Om den hittar ett element i fel ordning direkt så behöver den bara en lookup, alltså $O(1)$.

Förstå kod: tidskomplexitet

Fråga: Vilken tidskomplexitet har koden? (VT22)

```
public static int compute(int n)
{
    if (n < 10)
        return n;
    else
    {
        int number = 0;
        for (int i = 1; i < 1000; i = i * 2)
        {
            number += i;
        }
        return number;
    }
}
```

- Oavsett hur stort n är kommer loopen bara att gå max 1000 varv
- **Metoden har därför tidskomplexiteten $O(1)$:** antalet operationer som utförs förändras inte i proportion till indatan (utom vid väldigt små värden, men vi bryr oss ju bara om när n är stort)

Skriva kod (VT23)

1. Implementera metoden så den fungerar enligt JavaDoc-kommentaren.
2. Ange din lösnings tidskomplexitet i en eller flera kommentarer (där n är längden på a).

Förutom att vara korrekt ska din implementation ska vara så effektiv som möjligt (hellre $O(n \cdot \log n)$ än $O(n^2)$ osv). Du väljer om du vill skriva funktionaliteten själv eller om du vill använda datastrukturer och/eller algoritmer från Javas API:er. Koden behöver inte vara exakt korrekt syntaxmässigt (om du t ex inte kommer ihåg vad en metod heter). Det viktiga är att vi kan se att du förstår hur metoden kan implementeras.

```
/**
 * Returns true if there is one or more duplicate elements in an unsorted array.
 * If a contains [ 2, 5, 3, 2 ] the method will return true since it contains
 * duplicate elements with value 2.
 *
 * @param a an unsorted array
 * @return true if there are duplicates in the array, false if not
 */
public boolean hasDuplicateElements(int[] a) { }
```

Lösning: Sortera arrayen

- Börja med att fundera på vilket det **lättaste sättet** är **att kolla kopior**: ganska snabbt inser man att det är **om man har en sorterad samling**
- Om **två tal intill varandra** är **samma tal** innehåller samlingen kopior, och vi kan returnera true direkt i loopen, och är då klara. Annars går loopen färdigt och vi returnerar false.
- **Vi tar in en array** i metoden, så vi kan använda **Arrays.sort()** för att sortera den (använd Javas inbyggda API:er - **gör det inte svårare** för er själva än ni behöver!)
- **Arrays.sort()** använder QuickSort som **är $O(n \cdot \log n)$** både i bästa fallet och i genomsnitt. Värsta fallet är **$O(n^2)$** , men den är optimerad för att undvika det
- Vi kan nu fylla i metoden:

Lösning: Sortera arrayen

```
import java.util.Arrays;

/**
 * Returns true if there is one or more duplicate elements in an unsorted array.
 * If a contains [ 2, 5, 3, 2 ] the method will return true since it contains
 * duplicate elements with value 2.
 *
 * @param a an unsorted array
 * @return true if there are duplicates in the array, false if not
 */
public boolean hasDuplicateElements(int[] a)
{
    Arrays.sort(a);

    for (int i = 0; i < a.length - 1; i++)
    {
        if(a[i] == a[i+1]) return true;
    }

    return false;
}
```

Lösning: Sortera arrayen

```
public boolean hasDuplicateElements(int[] a)
{
    Arrays.sort(a);                // O(n*logn)

    for (int i = 0; i < a.length - 1; i++)    // O(n)
    {
        if(a[i] == a[i+1]) return true;        // O(1)
    }

    return false;                  // O(1)
}
```

$$O(n \cdot \log n) + (\cancel{O(n)} * \cancel{O(1)}) + \cancel{O(1)} = O(n \cdot \log n)$$

Alternativ lösning: ett Set

```
import java.util.HashSet;

/**
 * Returns true if there is one or more duplicate elements in an unsorted array.
 * If a contains [ 2, 5, 3, 2 ] the method will return true since it contains
 * duplicate elements with value 2.
 *
 * @param a an unsorted array
 * @return true if there are duplicates in the array, false if not
 */
public boolean hasDuplicateElements(int[] a)
{
    Set<Integer> hashSet = new HashSet<>();

    for (int number : a)
    {
        if(!hashSet.add(number)) return true;           //Om vi inte kan lägga till numret
                                                         //finns det redan i setet
    }

    return false;
}
```

Alternativ lösning: ett Set

```
public boolean hasDuplicateElements(int[] a)
{
    HashSet<Integer> hashset = new HashSet<>();    //O(1)

    for (int number : a)                          //O(n)
    {
        if(!hashSet.add(number)) return true;    //O(1)
    }

    return false;                                //O(1)
}
```

$$O(\cancel{1}) + (O(n) * O(\cancel{1})) + O(\cancel{1}) = O(n)$$

- HashSet returnerar false vid insättning av dubletter, så vi vet direkt om värdet redan finns eller ej
- Vi använder HashSet och inte HashMap eftersom vi bara har vanliga heltal: maps är för nyckel-värde-par
- Kan också använda TreeSet för $O(n \cdot \log n)$ -lösning

Kommentera och förstå kod (VT24)

Du ska utifrån koden nedan:

1. Ändra namn på metoden och variabler så de får tydliga och informativa namn. (2 p)
2. Färdigställa JavaDoc-kommentaren så den blir tydlig, informativ och korrekt. (2 p)
3. Ange vilken tidskomplexitet metoden har i bästa och värsta fall (där n är längden på l). Du ska även motivera varför metoden har denna tidskomplexitet. (2 p)

```
/**
 *
 * @param
 * @return
 */
public LinkedList<Integer> method(LinkedList<Integer> l) {
    Stack<Integer> s = new Stack<>();
    for (int i : l) {
        s.push(i);
    }
    LinkedList<Integer> a = new LinkedList<>();
    While (!s.empty()) {
        a.add(s.pop());
    }
    return a;
}
```

Kommentera och förstå kod (VT24)

Du ska:

Utifrån koden nedan:

1. Ändra namn på metoden och variabler så de får tydliga och informativa namn. (2 p)

```
/**
 *
 * @param originalList
 * @return
 */
public LinkedList<Integer> method reverseList(LinkedList<Integer> l originalList) {
    Stack<Integer> s stack = new Stack<>();
    for (int i item : l originalList) {
        stack.push(item);
    }
    LinkedList<Integer> a reversedList = new LinkedList<>();
    while (!stack.empty()) {
        reversedList.add(stack.pop());
    }
    return reversedList;
}
```

Kommentera och förstå kod (VT24)

2. Färdigställa JavaDoc-kommentaren så den blir tydlig, informativ och korrekt. (2 p)

```
/**
 * Reverse the order of items in a list
 * @param originalList the original list
 * @return the list in a reversed order
 */
public LinkedList<Integer> reverseList(LinkedList<Integer> originalList) {
    Stack<Integer> stack = new Stack<>();
    for (int item : originalList) {
        stack.push(item);
    }
    LinkedList<Integer> reversedList = new LinkedList<>();
    while (!stack.empty()) {
        reversedList.add(stack.pop());
    }
    return reversedList;
}
```

Kommentera och förstå kod (VT24)

3. Ange vilken tidskomplexitet metoden har i bästa och värsta fall (där n är längden på l). Du ska även motivera varför metoden har denna tidskomplexitet. (2 p)

```
/**
 * Reverse the order of items in a list
 * @param originalList the original list
 * @return the list in a reversed order
 */
public LinkedList<Integer> reverseList(LinkedList<Integer> originalList) {
    Stack<Integer> stack = new Stack<>(); //O(1)
    for (int item : originalList) { //O(n)
        stack.push(item); //O(1) item läggs alltid högst upp
    }
    LinkedList<Integer> reversedList = new LinkedList<>(); //O(1)
    while (!stack.empty()) { //O(n)*O(1) = O(n) kolla om stacken är tom för varje n
        reversedList.add(stack.pop()); //O(1) + O(1) = O(1) poppa det översta elementet från
        stacken O(1) och lägg till i det på slutet av den länkade listan O(1) eftersom en länkad
        lista i Java som standard är dubbellänkad
    }
    return reversedList; // O(1)
} // sammanlagt O(1) + O(n*1) + O(1) + O(n*1) + O(1) = O(n)
```

Välja sorteringsalgoritm (VT24)

Du ska:

utveckla en applikation som behöver sortera listor med stora mängder data. Listorna är i ca 50 % av fallen redan sorterade men för att vara säker behöver vi varje gång antingen sortera om listorna eller kontrollera om de är osorterade och i så fall sortera dem. I de fall listorna är osorterade är det maximalt 15 % av elementen som inte är i ordning. Utifrån scenariot:

1. Väljer du att varje gång sortera listorna eller istället att kontrollera om de är osorterade och i så fall sortera dem? (5 p)
2. Vilken sorteringsalgoritm väljer du? (5 p)

För båda frågorna behöver du motivera ditt svar genom att resonera om dina val jämfört med andra möjliga val. I ditt resonemang vill vi att du hänvisar till tidskomplexitet. Om du gör antaganden vill vi att du tydligt beskriver dessa.

Välja sorteringsalgoritm (VT24)

1. Väljer du att varje gång sortera listorna eller istället att kontrollera om de är osorterade och i så fall sortera dem? (5 p)

Jag skulle kolla om varje lista är sorterad en gång först för att sedan **spara in på 50% av sorteringarna**. Att kontrollera om en lista är sorterad eller ej kan göras i $O(n)$ tid, och vi behöver då inte göra någon sortering alls.

I de 50% av fallen där listan inte är sorterad kan vi därefter göra en sortering, och den inledande checken kommer då inte att påverka tidskomplexiteten.

Välja sorteringsalgoritm (VT24)

2. Vilken sorteringsalgoritm väljer du? (5 p)

Insertion Sort har vinst när **stor mängd data redan är sorterad** så den väljs. Mergesort skulle i och för sig passa bra när det är mycket osorterad data, men den är inte in-place, och den presterar alltid $O(n \cdot \log n)$ tid.

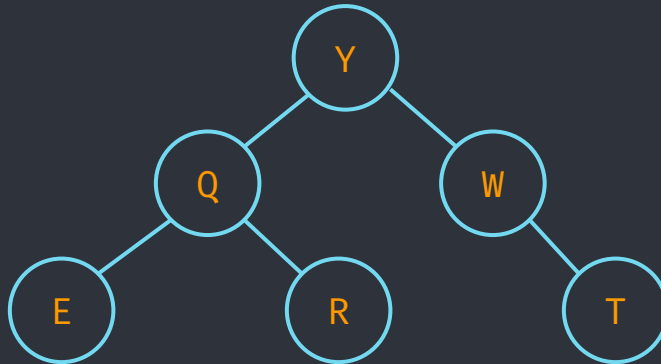
Insertion Sort har $O(n)$ när elementen redan är sorterade eftersom den bara flyttar ett element om det är felplacerat. I värsta fall är den dock $O(n^2)$ om hela listan är helt osorterad. I detta fall kommer det värsta fallet bara att inträffa för 15 procent av elementen. Detta medför att Insertion Sort kommer att prestera nära $O(n)$, eller som mest $O(n + k \cdot n)$ där $k \approx 0,15n$.

Att den är in-place betyder att den inte behöver någon ytterligare datastruktur under sorteringen vilket sparar minne. Detta låter önskvärt i och med att det står att listorna innehåller stora mängder av data.

Träd (VT23)

Utifrån det binära trädet (figuren):

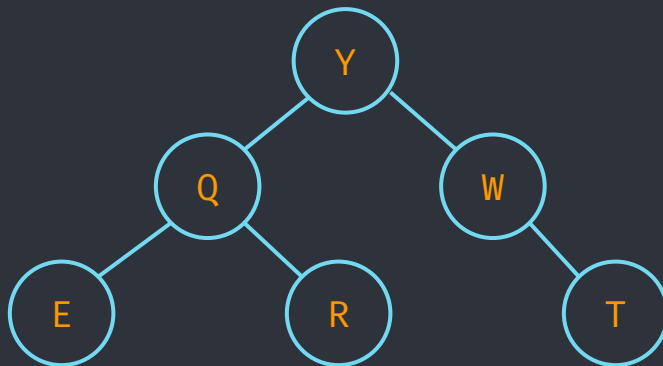
1. I vilken ordning kommer noderna besökas vid en post-order-traversering?
2. I vilken ordning kommer noderna besökas vid en pre-order-traversering



Träd (VT23)

1. I vilken ordning kommer noderna besökas vid en post-order-traversering?

Post-order = Vänsternod-Högernod-Rotnod (VHR): roten besöks sist

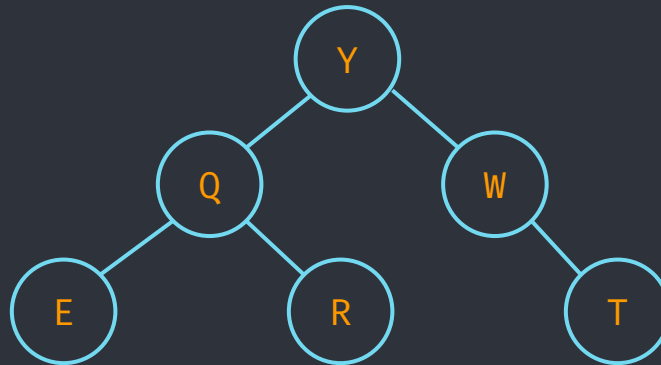


Svar: E R Q T W Y

Träd (VT23)

2. I vilken ordning kommer noderna besökas vid en pre-order-traversering

Pre-order = vi processerar rotnoden först: Rotnod-Vänsternod-Högerod (RVH)



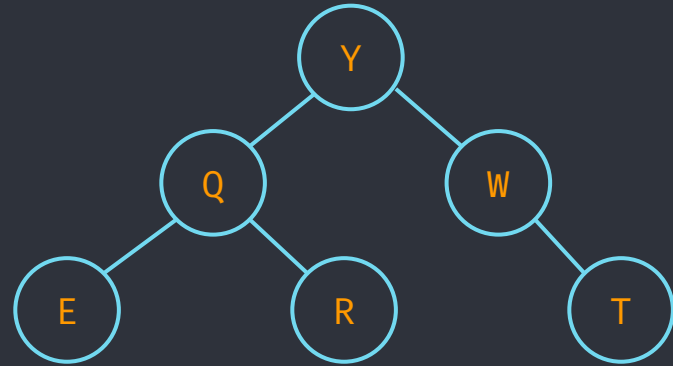
Svar: Y Q E R W T

Träd (VT24)

Utifrån trädet och koden nedan:

1. I vilken ordning kommer noderna skrivas ut om vi skickar in noden Y till metoden? (2 p)
2. I vilken ordning kommer noderna skrivas ut om vi skickar in noden Q till metoden? (2 p)
3. Vad kallas traverseringen som koden medför? (2 p)

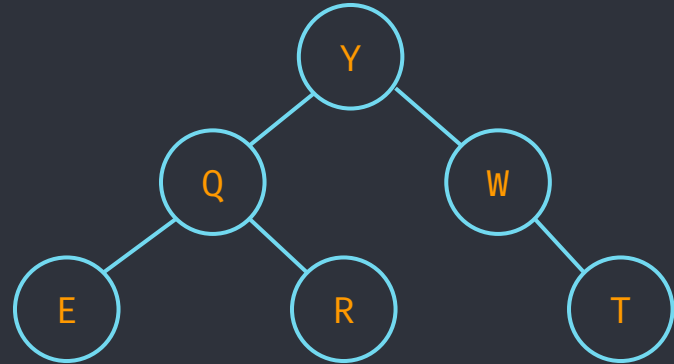
```
public void visit(TreeNode node)
{
    if(node != null)
    {
        System.out.println(node.getName());
        visit(node.getLeftChild());
        visit(node.getRightChild());
    }
}
```



Träd (VT24)

1. I vilken ordning kommer noderna skrivas ut om vi skickar in noden Y till metoden? (2 p)

```
public void visit(TreeNode node)
{
    if(node != null)
    {
        System.out.println(node.getName());
        visit(node.getLeftChild());
        visit(node.getRightChild());
    }
}
```

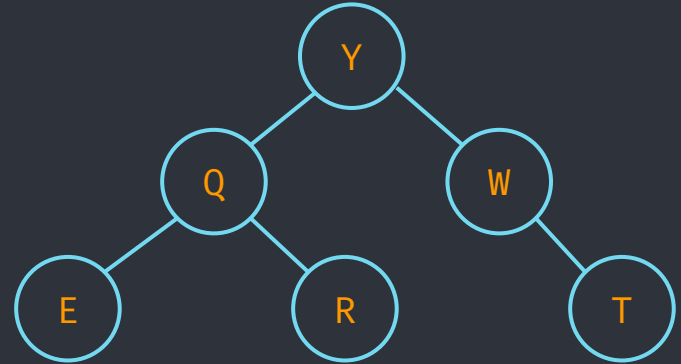


Svar: Y Q E R W T

Träd (VT24)

2. I vilken ordning kommer noderna skrivas ut om vi skickar in noden Q till metoden? (2 p)

```
public void visit(TreeNode node)
{
    if(node != null)
    {
        System.out.println(node.getName());
        visit(node.getLeftChild());
        visit(node.getRightChild());
    }
}
```

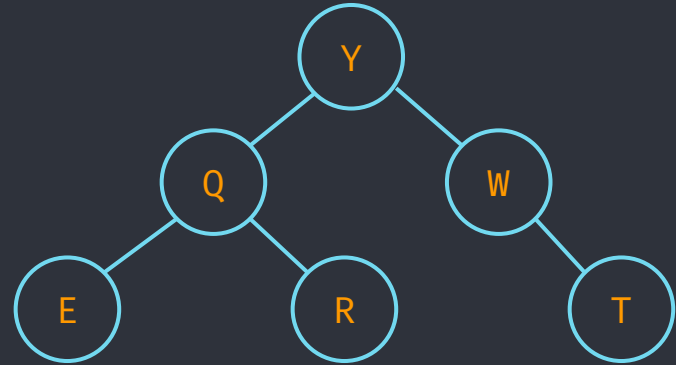


Svar: Q E R

Träd (VT24)

3. Vad kallas traverseringen som koden medför? (2 p)

```
public void visit(TreeNode node)
{
    if(node != null)
    {
        System.out.println(node.getName());
        visit(node.getLeftChild());
        visit(node.getRightChild());
    }
}
```

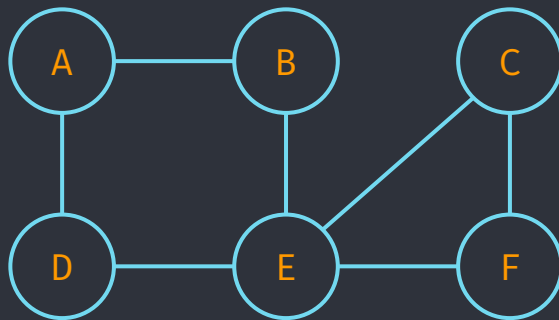


Svar: Pre-Order, eftersom vi besöker rotnoden först (pre = före)

Grafer (VT23)

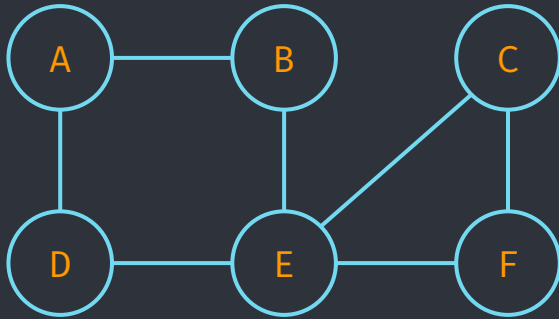
Utifrån grafen (figuren):

1. Skapa en grannmatrix (Adjacency Matrix).
2. I vilken ordning kommer noderna besökas om vi startar från C och använder en djupet-först-sökning (Depth First Search)? I ditt svar ska du utgå från att alla noder lagrar vägen till andra noder i bokstavsordning.



Grafer (VT23)

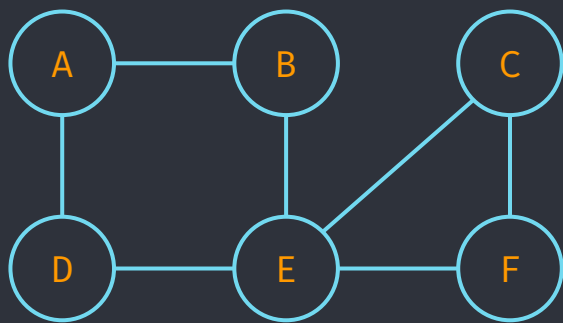
1. Skapa en grannmatris (Adjacency Matrix).



	A	B	C	D	E	F
A	0	1	0	1	0	0
B	1	0	0	0	1	0
C	0	0	0	0	1	1
D	1	0	0	0	1	0
E	0	1	1	1	0	1
F	0	0	1	0	1	0

Grafer (VT23)

2. I vilken ordning kommer noderna besökas om vi startar från C och använder en djupet-först-sökning (Depth First Search)? I ditt svar ska du utgå från att alla noder lagrar vägen till andra noder i bokstavsordning.



Djupet först: använd en stack

Svar: C E B A D F

Välj datastruktur (VT23)

Scenario:

Du ska utveckla en applikation som behöver lagra mycket data. När applikationen används kommer den *ofta* behöva lista alla element, *sällan* lägga till och ta bort element och *ofta* söka efter enskilda element.

Utifrån scenariot:

Vilken datastruktur väljer du för att lagra data i applikationen?

Du behöver motivera ditt svar genom att resonera om dina val jämfört med andra möjliga val. I ditt resonemang vill vi att du hänvisar till tidskomplexitet. Om du gör antaganden vill vi att du tydligt beskriver dessa.

Välj datastruktur (VT23)

	Lista Samtliga	Lägga till	Ta bort	Söka	Intervall
Array	$O(n)$	$O(n)$	$O(n)$	$O(n)$	$O(k)$
ArrayList	$O(n)$	$O(1)$	$O(n)$	$O(n)$	$O(k)$
LinkedList	$O(n)$	$O(1)$	$O(1)$	$O(n)$	$(n+k)$
TreeMap	$O(n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(1)$
HashMap	$O(n)$	$O(1)$	$O(1)$	$O(1)$	$O(k)$

Antagande: Eftersom uppgiften säger att vi ofta ska lista samtliga element tänker jag att vi vill kunna göra det i sorterad ordning.

En HashMap är inte sorterad, men en TreeMap är.

Välj datastruktur (VT23)

Svar:

Jag skulle välja en TreeMap. Om vi antar att datan lagras i form av nyckel-värde-par så känns en map lämplig. HashMap har $O(1)$ på uthämtning, insättning och borttagning, men den är inte sorterad. Om vi itererar över den kommer datan att skrivas ut huller om buller.

En TreeMap sorterar datan vid insättning och är fortfarande väldigt snabb: Den har $O(\log n)$ på uthämtning, insättning och borttagning av element.

Den har också fördelen att vi kan lagra datan fragmentariskt i minnet och behöver inte allokera ett stort block sammanhängande RAM. Eftersom det stod i uppgiften att vi ska lagra *mycket data* kan det här vara en annan viktig aspekt.

Tidskomplexitet och algoritmdesignsteknik (VT25)

Utifrån koden nedan:

1. Vad har metoden för tidskomplexitet (där n är antalet element i det binära trädet) i bästa och värsta fallet? Motivera ditt svar. (4 p)

2. Vilken algoritmdesignsteknik representerar metoden? (2 p)

```
/**
 * Returns the value to which the key is mapped or null
 * if this binary tree map contains no mapping for the key.
 * @param key the key
 * @return value
 */
public Integer get(int key)
{
    Node node = root;

    while(node != null)
    {
        if(node.getValue() > key) { node = node.left; }
        else if(node.getValue() < key) { node = node.right; }
        else { return node.value; }
    }
    return null;
}
```

- Metoden är en sökning i ett binärt träd
- Om nyckeln är mindre: gå till vänster; Om nyckeln är större: gå till höger
- Stoppar när den hittat värdet eller om en nod är null
- Divide-and-conquer-teknik: vi halverar trädet varje gång vi gör ett val
- $O(1)$ i bästa fall (värdet är alltid först, $O(\log n)$ i värsta fall (om trädet är balanserat)
- Obalanserat träd i värsta fall $O(n)$